

```

import os
import shutil
import numpy as np
import cv2
from tqdm import tqdm
from sklearn.model_selection import train_test_split, KFold
from sklearn.utils import shuffle
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dropout, Dense

# ----- DATA SPLITTING -----
dataset_folder = "/content/drive/MyDrive/Top View Pics"
output_train_folder = "/content/drive/My Drive/output/train"
output_test_folder = "/content/drive/My Drive/output/test"
labels = ['AD Top View', 'CN Top View', 'MCI Top View']

os.makedirs(output_train_folder, exist_ok=True)
os.makedirs(output_test_folder, exist_ok=True)

for label in labels:
    section_path = os.path.join(dataset_folder, label)
    if not os.path.isdir(section_path): continue
    files = [f for f in os.listdir(section_path) if os.path.isfile(os.path.join(section_path, f))]
    train_files, test_files = train_test_split(files, test_size=0.2, random_state=42)
    train_section_path = os.path.join(output_train_folder, label)
    test_section_path = os.path.join(output_test_folder, label)
    os.makedirs(train_section_path, exist_ok=True)
    os.makedirs(test_section_path, exist_ok=True)
    for f in train_files: shutil.copy(os.path.join(section_path, f), os.path.join(train_section_path, f))
    for f in test_files: shutil.copy(os.path.join(section_path, f), os.path.join(test_section_path, f))

# ----- IMAGE LOADING -----
def load_images(folder_path, label, data_list, label_list):
    for image_file in tqdm(os.listdir(folder_path), desc=f"Loading {label}"):
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path)
        if image is not None:
            image = cv2.resize(image, (150, 150))
            data_list.append(image)
            label_list.append(label)

X_train, Y_train, X_test, Y_test = [], [], [], []
for label in labels:
    train_folder = os.path.join(train_path := output_train_folder, label)
    test_folder = os.path.join(test_path := output_test_folder, label)
    if os.path.exists(train_folder): load_images(train_folder, label, X_train, Y_train)
    if os.path.exists(test_folder): load_images(test_folder, label, X_test, Y_test)

X_train, Y_train = shuffle(np.array(X_train), np.array(Y_train), random_state=42)
X_test, Y_test = np.array(X_test), np.array(Y_test)

label_to_int = {label: i for i, label in enumerate(labels)}
Y_train_int = np.array([label_to_int[y] for y in Y_train])
Y_test_int = np.array([label_to_int[y] for y in Y_test])
Y_train_onehot = to_categorical(Y_train_int, 3)
Y_test_onehot = to_categorical(Y_test_int, 3)

Loading AD Top View: 100% |██████████| 508/508 [00:04<00:00, 117.86it/s]
Loading AD Top View: 100% |██████████| 128/128 [00:01<00:00, 126.53it/s]
Loading CN Top View: 100% |██████████| 1075/1075 [00:12<00:00, 89.23it/s]
Loading CN Top View: 100% |██████████| 269/269 [00:02<00:00, 115.83it/s]
Loading MCI Top View: 100% |██████████| 1152/1152 [00:15<00:00, 72.72it/s]
Loading MCI Top View: 100% |██████████| 288/288 [00:02<00:00, 125.28it/s]

# ----- MODEL DEFINITION -----
base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=(150, 150, 3))
x = GlobalAveragePooling2D()(base_model.output)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=output)

```

```

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# ----- TRAINING -----
history = model.fit(X_train, Y_train_onehot, validation_data=(X_test, Y_test_onehot), epochs=10, batch_size=32, verbose=1)

# ----- EVALUATION -----
test_loss, test_accuracy = model.evaluate(X_test, Y_test_onehot)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

pred_probs = model.predict(X_test)
pred_labels = np.argmax(pred_probs, axis=1)
actual_labels = np.argmax(Y_test_onehot, axis=1)

print(classification_report(actual_labels, pred_labels, target_names=['AD', 'CN', 'MCI']))

# ----- CONFUSION MATRIX -----
cm = confusion_matrix(actual_labels, pred_labels)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='YlOrBr', xticklabels=['AD', 'CN', 'MCI'], yticklabels=['AD', 'CN', 'MCI'])
plt.title("Confusion Matrix - DenseNet")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig('confusion_matrix_yellow.png', dpi=300)
plt.show()

# ----- MODEL ACCURACY AND LOSS -----
plt.figure(figsize=(14, 5))

# Accuracy Plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], 'bo-', label='Train Accuracy')
plt.plot(history.history['val_accuracy'], 'ro-', label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Loss Plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], 'bo-', label='Train Loss')
plt.plot(history.history['val_loss'], 'ro-', label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig('accuracy_loss.png', dpi=300)
plt.show()

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_ordering_tf_29084464/29084464 0s 0us/step

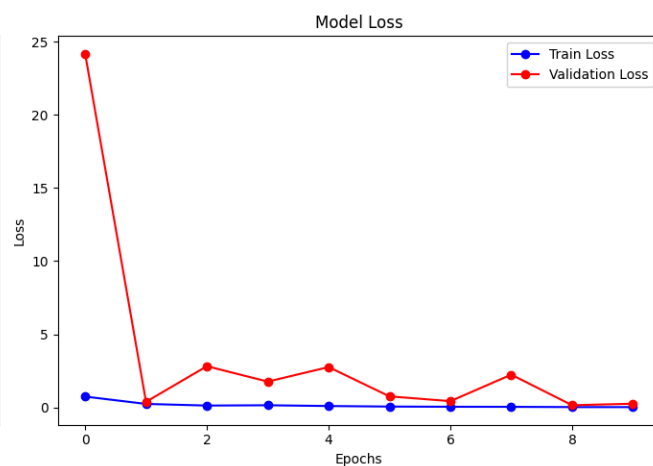
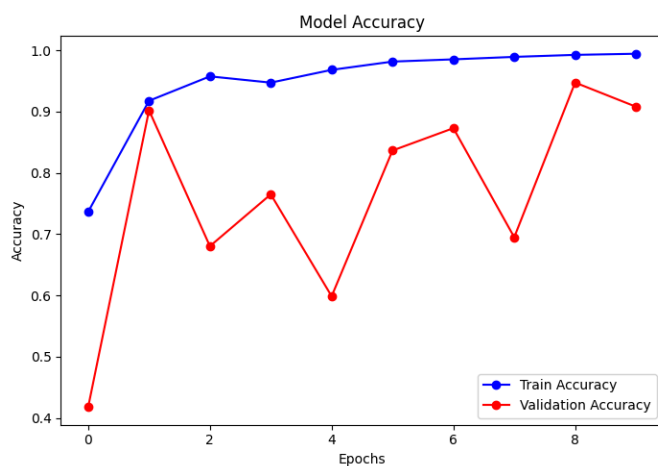
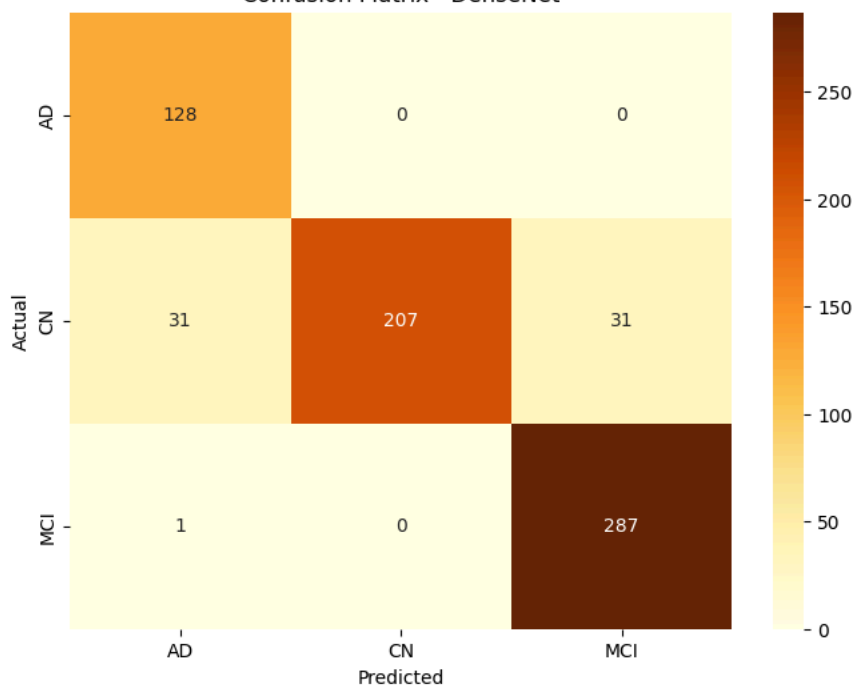
```
Epoch 1/10
86/86 986s 10s/step - accuracy: 0.6305 - loss: 1.0580 - val_accuracy: 0.4175 - val_loss: 24.1837
Epoch 2/10
86/86 896s 10s/step - accuracy: 0.9046 - loss: 0.2614 - val_accuracy: 0.9022 - val_loss: 0.3796
Epoch 3/10
86/86 884s 10s/step - accuracy: 0.9476 - loss: 0.1358 - val_accuracy: 0.6803 - val_loss: 2.8166
Epoch 4/10
86/86 857s 10s/step - accuracy: 0.9541 - loss: 0.1287 - val_accuracy: 0.7650 - val_loss: 1.7629
Epoch 5/10
86/86 898s 10s/step - accuracy: 0.9617 - loss: 0.1116 - val_accuracy: 0.5985 - val_loss: 2.7568
Epoch 6/10
86/86 923s 10s/step - accuracy: 0.9836 - loss: 0.0494 - val_accuracy: 0.8365 - val_loss: 0.7528
Epoch 7/10
86/86 925s 10s/step - accuracy: 0.9853 - loss: 0.0456 - val_accuracy: 0.8730 - val_loss: 0.4267
Epoch 8/10
86/86 922s 10s/step - accuracy: 0.9889 - loss: 0.0395 - val_accuracy: 0.6949 - val_loss: 2.2372
Epoch 9/10
86/86 919s 10s/step - accuracy: 0.9938 - loss: 0.0192 - val_accuracy: 0.9474 - val_loss: 0.1485
Epoch 10/10
86/86 889s 10s/step - accuracy: 0.9947 - loss: 0.0145 - val_accuracy: 0.9080 - val_loss: 0.2501
22/22 42s 2s/step - accuracy: 0.9100 - loss: 0.2531
```

Test Loss: 0.2501

Test Accuracy: 90.80%

	precision	recall	f1-score	support
AD	0.80	1.00	0.89	128
CN	1.00	0.77	0.87	269
MCI	0.90	1.00	0.95	288
accuracy			0.91	685
macro avg	0.90	0.92	0.90	685
weighted avg	0.92	0.91	0.91	685

Confusion Matrix - DenseNet



```

# ----- CROSS VALIDATION ACCURACY -----
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv accuracies = []

for train_index, val_index in kf.split(X_train):
    X_cv_train, X_cv_val = X_train[train_index], X_train[val_index]
    Y_cv_train, Y_cv_val = Y_train_onehot[train_index], Y_train_onehot[val_index]

    base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=(150, 150, 3))
    x = GlobalAveragePooling2D()(base_model.output)
    x = Dropout(0.5)(x)
    output = Dense(3, activation='softmax')(x)
    cv_model = Model(inputs=base_model.input, outputs=output)
    cv_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

    cv_model.fit(X_cv_train, Y_cv_train, epochs=3, batch_size=32, verbose=0)
    _, acc = cv_model.evaluate(X_cv_val, Y_cv_val, verbose=0)
    cv accuracies.append(acc)

# Plot cross-validation results
plt.figure(figsize=(6, 4))
plt.bar(range(1, 6), [a * 100 for a in cv accuracies], color='goldenrod')
plt.title('Cross Validation Accuracy per Fold')
plt.xlabel('Fold')
plt.ylabel('Accuracy (%)')
plt.ylim(0, 100)

```